```
        FLOAT64,
        NULL
    }
}
```

B.5.6. optional

optional Qualifier Syntax

```
...
field-name [ optional ] : type-or-ref,
...
```

The `optional` qualifier declares that a field within a `STRUCTURE` or `FIELD GROUP` is optional, and MAY be omitted by an encoder. The `optional` qualifier MAY only appear on the name portion of a field definition within either a `STRUCTURE` or `FIELD GROUP`.

Note that an optional field is distinct, both semantically and in terms of encoding, from a field whose type has been declared nullable. In the former case the field MAY be omitted from the encoding altogether. In the latter case the field SHALL appear within the encoding, however its value MAY be encoded as a TLV Null. It is legal to declare a field that is both `optional` and `nullable`.

The conditions under which an optional field can be omitted depend on the semantics of the structure. In some cases, fields MAY be omitted entirely at the discretion of the sender. In other cases, omission of a field MAY be contingent on the value present in another field. In all cases, prose documentation associated with the field definition SHALL make clear the rules for when the field may be omitted.

Optional fields are allowed within `FIELD GROUP` and retain their optionality when included within `STRUCTURE`.

optional Qualifier Example

```
user-information => STRUCTURE [ extensible ]
{
    user-id [1]                 : UNSIGNED INTEGER,
    first-name [2]              : STRING,
    middle-name [3, optional]   : STRING,         // MIGHT be omitted
    last-name [4]               : STRING,
    email-address [5, optional] : STRING,         // MIGHT be omitted
}
```

B.5.7. range